

université BORDEAUX	ANNÉE UNIVERSITAIRE 2025 - 2026 SESSION 1 DE PRINTEMPS 2026	Collège Sciences et Technologie Masters
	MENTION : BIOINFORMATIQUE Code UE : 4TBI808 Intitulé de l'épreuve : Nouvelles générations de séquençage Date : 28/04/2026 Heure : 11h30 Durée : 1h30 Documents : non autorisés Épreuve de Mme Raluca Uricaru.	

1 Recherche exacte de motifs (13 pts)

Question 1.1 Quelle est la différence entre une méthode de recherche exacte de motifs *online* et une méthode *offline* ? Donnez au moins deux exemples de méthodes dans chaque catégorie. Dans le contexte de l'alignement d'un grand nombre de lectures sur un génome de référence, quel type d'approche est le plus pertinent ? Justifiez.

Question 1.2 On considère la liste lexicographiquement triée suivante de *3-mers* :

[ACG, ACG, ACT, AGT, CCA, CCG, CGT, CGT, TAA]

1. Recherchez le motif *CGT* par *recherche dichotomique*. Donnez les indices testés successivement.
2. Une fois une occurrence trouvée, expliquez comment retrouver *toutes* les occurrences de *CGT* dans la liste.
3. Recherchez de la même manière le motif *AAA*. Que concluez-vous ?
4. Quelle est la complexité temporelle de la recherche dichotomique dans une liste triée de taille n ?
5. En quoi cette stratégie de recherche dans une liste triée est-elle liée au principe de recherche dans une *table des suffixes* ?

Question 1.3 À quoi servent les *minimizers* dans le contexte de la recherche de motifs en bioinformatique ? Expliquez brièvement leur principe et leur utilité.

Question 1.4 On donne le texte $t = \text{TACTA\$}$.

1. Construisez la table des suffixes (*suffix array*) de t .
2. Déduisez-en la transformée de Burrows-Wheeler (*BWT*) de t .
3. Expliquez en quelques phrases pourquoi la *BWT* est utile pour la recherche de motifs.

Question 1.5 On donne un code Python qui construit un arbre des suffixes de manière incrémentale. On applique ce code au texte *GAGA*.

1. Donnez la liste des suffixes insérés successivement.
2. Pour chacun des suffixes, indiquez précisément ce que fait l'algorithme afin de l'insérer dans l'arbre : descente dans l'arbre, comparaison de caractères, création d'une feuille, éventuelle coupure d'une arête. Reliez chacune de ces étapes aux instructions ou blocs d'instructions correspondants dans le code fourni. Dessinez l'arbre obtenu après l'insertion de chaque suffixe.

3. Combien de feuilles l'arbre final contient-il ? À quoi correspondent-elles ?

```
1 def makeNode (label):
2     return {'lab' : label, 'out' : {}}
3
4 def buildST (s):
5     s += '$'
6     root = makeNode('root')
7     root['out'][s[0]] = makeNode(s)
8
9     for i in range(1, len(s)):
10        cur = root
11        j = i
12        while j < len(s):
13            if s[j] in cur['out']:
14                child = cur['out'][s[j]]
15                lab = child['lab']
16                k = j+1
17                while k-j < len(lab) and s[k] == lab[k-j]:
18                    k += 1
19                if k-j == len(lab):
20                    cur = child
21                    j = k
22            else:
23                cExist, cNew = lab[k-j], s[k]
24                mid = makeNode(lab[k-j])
25                mid['out'][cNew] = makeNode(s[k:])
26                mid['out'][cExist] = child
27                child['lab'] = lab[k-j:]
28                cur['out'][s[j]] = mid
29        else:
30            cur['out'][s[j]] = makeNode(s[j:])
31            break
32    return root
```

2 Graphes et assemblage (8 pts)

On considère les lectures suivantes :

ATGCA, TGCAT, GCATG, CATGG

Question 2.1 Construisez le graphe de De Bruijn pour $k = 3$.

1. Donnez l'ensemble des noeuds (ici les noeuds correspondent aux k -mers).
2. Donnez l'ensemble des arêtes (produites par les chevauchements de taille $k - 1$ entre les noeuds).
3. Dessinez le graphe obtenu.
4. Proposez une séquence reconstruite à partir d'un chemin dans ce graphe.

Question 2.2 Quel est l'effet du choix de la valeur de k dans un graphe de De Bruijn ?

Expliquez ce qui peut se passer lorsque k est :

1. trop petit ;
2. trop grand.

Question 2.3 Écrivez une fonction `all_kmers(reads, k)` qui prend en entrée une liste de lectures `reads` et un entier `k`, et qui renvoie la liste de tous les `k`-mers observés dans ces lectures. On autorise les doublons dans la liste renvoyée.

Exemple :

```
all_kmers(["ATGCG", "TGCGA"], 3)
doit renvoyer
["ATG", "TGC", "GCG", "TGC", "GCG", "CGA"]
```

Question 2.4 En faisant appel à la fonction `all_kmers(reads, k)` écrivez une nouvelle fonction `all_edges(reads, k)` qui prend en entrée une liste de lectures `reads` et un entier `k`, et qui renvoie la liste d'arêtes, chaque arête étant représentée par un couple (u, v) , où u et v sont des `k`-mers se chevauchant sur `k-1` caractères.

Exemple :

```
all_edges(["ATGCG", "TGCGA"], 3)
doit renvoyer
[("ATG", "TGC"), ("TGC", "GCG"), ("TGC", "GCG"), ("GCG", "CGA")]
```